UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Laboratorio de Compiladores

Práctica 6

Anexo: Ejercicios de Validación

Jesús Eliuth Martínez Mendoza

Octubre de 2026

Anexo: Ejercicios de Validación

1. Introducción

En este anexo se desarrolla la auditoría de una colección de estados LR(1) y su compactación LALR(1). El objetivo es identificar errores en la propagación de lookaheads, en la fusión de kernels y en la construcción de la tabla de análisis sintáctico. De acuerdo con el penúltimo dígito de la cuenta UNAM, que es impar, se utiliza la siguiente gramática:

$$\begin{aligned} S' &\to S \\ S &\to L = R \\ S &\to R \\ L &\to \&R \\ L &\to id \\ R &\to L \end{aligned}$$

2. Fase 1: Auditoría manual del caso base

2.1. Construcción del estado inicial I_0

Partimos del elemento inicial:

$$[S' \to \cdot S, \$]$$

Aplicando CLOSURE se incorporan las producciones de S. Para las producciones de L, los lookaheads se obtienen mediante FIRST(βa). En particular:

$$FIRST(= R\$) = \{=\}$$

y para la producción $S \to R$:

$$FIRST(\varepsilon\$) = \{\$\}$$

Al reunir los elementos correspondientes, el estado inicial queda:

$$I_0 = \left\{ \begin{array}{c} [S' \to \cdot S, \$] \\ [S \to \cdot L = R, \$] \\ [S \to \cdot R, \$] \\ [R \to \cdot L, \$] \\ [L \to \cdot \&R, \{=, \$\}] \\ [L \to \cdot id, \{=, \$\}] \end{array} \right\}$$

2.2. Construcción del estado I_2

Aplicamos la transición:

$$I_2 = GOTO(I_0, L)$$

Los elementos que tienen el punto antes de L avanzan una posición:

$$I_2 = \{[S \to L\cdot = R, \$],\ [R \to L\cdot, \$]\}$$

El primer elemento permite desplazar el terminal =, mientras que el segundo permite reducir mediante la producción $R \to L$ únicamente cuando el lookahead es \$.

2.3. Transición del operador unario

Calculamos:

$$GOTO(I_0, \&) = I_4$$

El punto avanza en los elementos correspondientes:

$$[L \to \& \cdot R, \{=, \$\}]$$

Aplicando CLOSURE, se incorpora:

$$[R \to \cdot L, \{=, \$\}]$$

Después se expanden las producciones de L:

$$I_4 = \left\{ \begin{array}{c} [L \to \& \cdot R, \{=, \$\}] \\ [R \to \cdot L, \{=, \$\}] \\ [L \to \cdot \& R, \{=, \$\}] \\ [L \to \cdot id, \{=, \$\}] \end{array} \right\}$$

2.4. Identificación de las tres inconsistencias

Error 1. Producción incorrecta en el estado inicial.

El caso base utiliza la producción $L \to *R$, aunque para esta variante debe utilizarse:

$$L \to \& R$$

Esto modifica la gramática que se está analizando.

Error 2. Omisión de la fusión de kernels equivalentes.

Los estados indicados en el caso base contienen el mismo núcleo LR(0), aunque sus lookaheads sean diferentes. La condición de fusión es:

$$CORE(I_i) = CORE(I_j)$$

Por lo tanto, deben fusionarse y combinar sus respectivos lookaheads. La diferencia entre los lookaheads no justifica mantenerlos separados.

Error 3. Reducción forzada en el estado I_2.

En el caso base se inserta una reducción mediante la producción 6 en la entrada correspondiente al terminal =. Sin embargo, los elementos del estado son:

$$[S \to L \cdot = R, \$] \qquad [R \to L \cdot, \$]$$

La reducción solamente está permitida con el lookahead \$. Para el terminal = corresponde realizar un desplazamiento. Por tanto:

$$ACTION[I_2, =] = shift \qquad ACTION[I_2, \$] = reduce(6)$$

2.5. Justificación formal

Los tres errores incumplen las condiciones de construcción de los autómatas LR(1) y LALR(1). El primero utiliza una producción diferente de la establecida para la variante parametrizada. El segundo no aplica correctamente la condición de igualdad de kernels para la compactación. El tercero ignora el lookahead específico de un elemento LR(1) y asigna una reducción en una entrada donde corresponde un desplazamiento.

3. Fase 2: Inyección de nuevos errores

Para esta fase se introducen dos errores deliberados sobre el autómata correcto. Estos casos se mantienen separados del desarrollo original y se utilizan únicamente para analizar las consecuencias de una construcción incorrecta.

3.1. Fallo tipo A: Propagación incorrecta de lookahead

Consideramos el estado obtenido mediante:

$$GOTO(I_0, \&) = I_4$$

Durante su cerradura aparece el elemento:

$$[R \to \cdot L, \{=, \$\}]$$

Para expandir las producciones de L, aplicamos la regla:

$$b \in FIRST(\beta a)$$

Como después de L no existe ningún símbolo adicional:

$$\beta = \varepsilon$$

Por tanto:

$$FIRST(\varepsilon\{=, \$\}) = \{=, \$\}$$

Los elementos correctos son:

$$[L \to \cdot \& R, \{=, \$\}] \qquad [L \to \cdot id, \{=, \$\}]$$

Error introducido deliberadamente:

Se asigna el lookahead & al elemento:

$$[L \to \cdot id, \&]$$

Esta asignación es incorrecta porque & no pertenece al resultado de FIRST(βa). Si posteriormente se realiza la transición con id, el elemento completo tendría un lookahead incorrecto y podría introducir una acción de reducción en una entrada que no corresponde.

Corrección: recalcular la cerradura mediante FIRST(βa) y conservar únicamente los lookaheads = y $.

3.2. Fallo tipo B: Fusión inválida de kernels

Consideramos los siguientes estados:

$$I_5 = \{[L \to id\cdot, \{=, \$\}]\}$$

$$I_7 = \{[L \to \& R\cdot, \{=, \$\}]\}$$

Se introduce deliberadamente la fusión incorrecta:

$$J_{falso} = I_5 \cup I_7$$

Para comprobar si la fusión es válida, eliminamos los lookaheads. Los núcleos son:

$$CORE(I_5) = \{L \to id\cdot\}$$

$$CORE(I_7) = \{L \to \& R\cdot\}$$

Como:

$$CORE(I_5) \neq CORE(I_7)$$

la fusión no está permitida.

Consecuencia del error: el estado artificial contendría dos elementos completos:

$$[L \to id\cdot, \{=, \$\}] \qquad [L \to \&R\cdot, \{=, \$\}]$$

Para el terminal = aparecerían dos reducciones: $L \to id$ y $L \to \&R$. Lo mismo sucedería para \$. Esto produciría un conflicto Reduce/Reduce artificial.

Corrección: mantener los estados separados, porque sus núcleos LR(0) son diferentes.

4. Fase 3: Contraste crítico y experimentación con prompts

Para evaluar distintas formas de utilizar un modelo de lenguaje durante la auditoría, se realizaron tres consultas con los mismos errores deliberados.

4.1. P1: Prompt directo

Se solicitó identificar los errores presentes en los estados LR(1) y LALR(1), explicar las reglas incumplidas y proporcionar las correcciones. **Resultado:** identificó correctamente la asignación incorrecta del lookahead y la fusión inválida de los estados. La respuesta explicó que los lookaheads debían obtenerse mediante FIRST(βa) y que los kernels tenían que ser idénticos para permitir una fusión.

4.2. P2: Prompt estructurado

Se solicitó una auditoría formal que incluyera comprobaciones matemáticas, condiciones de los algoritmos, consecuencias en ACTION y correcciones. **Resultado:** identificó correctamente ambos errores y desarrolló con mayor detalle las operaciones involucradas. También explicó cómo la propagación incorrecta puede modificar las reducciones de ACTION y cómo la fusión inválida produce un conflicto Reduce/Reduce.

4.3. P3: Prompt socrático

Se solicitó que el modelo realizara preguntas progresivas sin revelar directamente las correcciones. Se trabajaron cinco etapas:

1. Identificación de β y a en el cálculo de FIRST(βa).
2. Identificación del origen de los lookaheads durante CLOSURE.
3. Diferenciación entre los elementos del kernel y los incorporados mediante CLOSURE.
4. Comprobación de la igualdad de kernels para permitir una fusión LALR(1).
5. Identificación y explicación del conflicto Reduce/Reduce.

Resultado: se respondieron correctamente las cinco etapas, obteniendo una evaluación de 5/5.

4.4. Cuadro comparativo

Criterio	P1 Directo	P2 Estructurado	P3 Socrático
Identificación de errores	Correcta	Correcta	Correcta
Justificación matemática	Breve	Detallada	Progresiva

Análisis de CLOSURE	Correcto	Correcto	Correcto
Análisis de kernels	Correcto	Correcto	Correcto
Análisis de ACTION	General	Detallado	Mediante preguntas
Omisiones	Poca explicación de consecuencias en ACTION.	Algunas explicaciones repetitivas.	No proporciona las respuestas directamente.
Alucinaciones	No documentadas	No documentadas	No documentadas
Principal ventaja	Rapidez	Profundidad y organización	Favorece el razonamiento

No se cuenta con transcripciones completas de las respuestas de P1, P2 y P3 para determinar si alguno de los modelos produjo información inventada. Por esa razón, las posibles alucinaciones se registran como no documentadas y no se afirma que estuvieron ausentes.

5. Reflexión crítica

Durante esta actividad pude comparar tres formas de utilizar un modelo de lenguaje para revisar errores en la construcción de autómatas LR(1) y LALR(1). Aunque los tres prompts permitieron identificar los dos fallos introducidos, cada uno presentó una manera diferente de abordar el problema.

El prompt directo (P1) fue el más rápido. Con una instrucción sencilla, el modelo identificó la propagación incorrecta del lookahead y la fusión inválida de los kernels. Sin embargo, sus explicaciones fueron más breves y no profundizaron tanto en las consecuencias de los errores sobre la tabla ACTION.

El prompt estructurado (P2) proporcionó una revisión más completa. Al solicitar comprobaciones matemáticas, condiciones del algoritmo y consecuencias, desarrolló con mayor detalle el cálculo de FIRST(βa), la comparación de los núcleos LR(0) y la aparición del conflicto Reduce/Reduce. Considero que fue el método más útil para obtener una explicación técnica organizada.

Por otra parte, el prompt socrático (P3) trabajó de una manera diferente. En lugar de proporcionar las respuestas directamente, realizó preguntas progresivas sobre los lookaheads, los kernels y las condiciones de fusión. Esto permitió revisar cada concepto por separado y relacionarlo con los errores planteados. Su principal desventaja fue que necesitó más interacciones para completar la revisión.

Al comparar los resultados, considero que el prompt estructurado fue el más completo para realizar una auditoría formal, mientras que el socrático fue más útil para repasar y comprender los conceptos. El directo funcionó como una primera revisión rápida.

Esta actividad también me permitió observar que no basta con aceptar una respuesta de un modelo de lenguaje. Es necesario comprobar sus operaciones y contrastarlas con las definiciones de los algoritmos. En particular, revisar el cálculo de FIRST(βa) y comprobar que los kernels sean idénticos antes de fusionar estados son pasos indispensables para evitar conclusiones incorrectas.

En conclusión, los tres métodos pueden ser útiles dependiendo del objetivo, pero la revisión matemática sigue siendo necesaria para validar los resultados.